Introduction to Core C++ Concepts

Chapter One

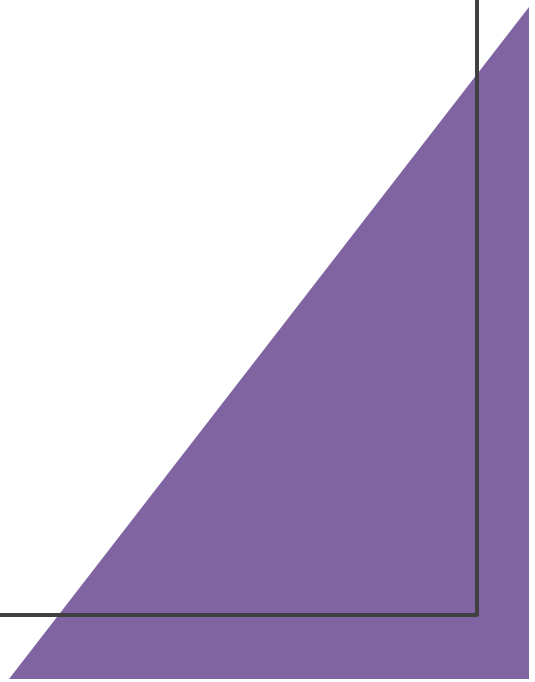
Piyush Pant

Overview of Topics

1.1 Structured vs Object Oriented Programming Paradigm

1.2 Features of Object Oriented Programming

1.3 Comparison on C and C++



Structured Programming

Structured Programming is a programming paradigm based on **functions and procedures**. The focus is on **writing a sequence of instructions** to manipulate data. It uses **control structures** (like loops, conditionals) and is usually **top-down** in design.

Key Characteristics:

- Emphasis on **functions** (not data).
- **Data and functions are separate.**
- Code is organized into procedures.
- No concept of objects or encapsulation.

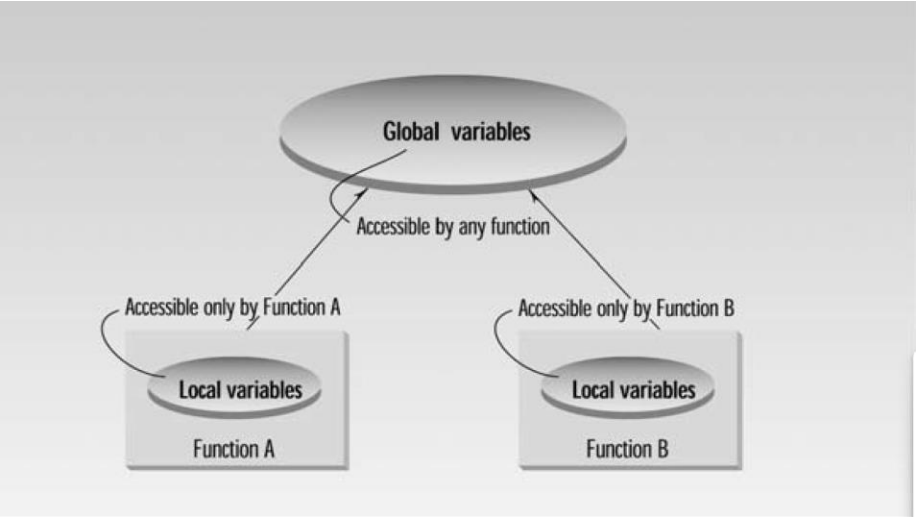
Problem with Structure Programming

Function have unrestricted access to global data.

Unrelated function and data

Problem with Structured Programming (eg : C)

- Global variables can be accessed and changed by **any function**
- There is **no protection** — even **unrelated functions** can modify important data
- This leads to **bugs**, **confusion**, and **no control over who changes what**
- C treats all functions as equals — no function “owns” any data



Example : C Programming

```
// C Program: Data and functions are separate
#include <stdio.h>
int balance = 0;  // Global data

// Function that changes the data
void deposit(int amount) {
    balance += amount;
}

// Function that uses the data
void showBalance() {
    printf("Balance: %d\n", balance);
}

int main() {
    deposit(1000);      // Function call
    showBalance();      // Output: Balance: 1000
    return 0;
}
```

- `balance` is just a global variable.
- Anyone can change it directly.
- `deposit()` and `showBalance()` are **not part of the data**.
- This structure is **prone to errors** as there's no protection for the data.

Object Oriented Programming

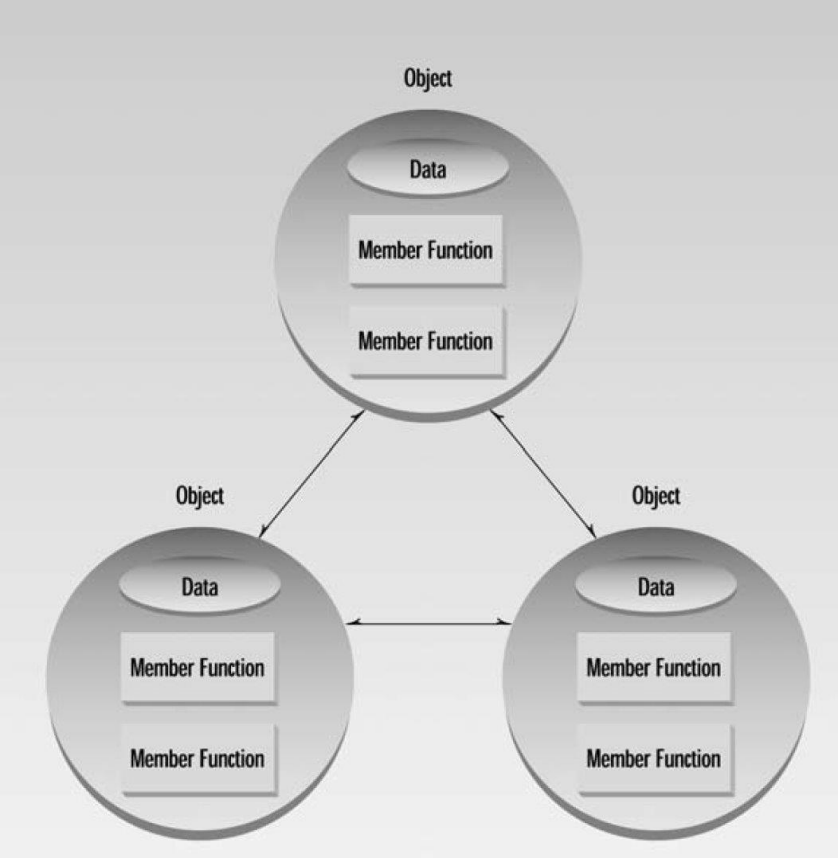
Object-Oriented Programming (OOP) is a programming paradigm that focuses on **objects** – combining **data and functions** into a single unit (class). It allows better **data protection**, **reusability**, and **modular design**.

Key Characteristics:

- Emphasis on **objects and classes**.
- **Data and functions are combined**.
- Supports **encapsulation**, **abstraction**, **inheritance**, and **polymorphism**.
- Bottom-up design. (Top-Down starts with the big picture and breaks it into smaller parts, while Bottom-Up builds small parts first and combines them into a complete system.)

OOP Paradigm

- Fundamental idea behind OOP language is to combine data and function operating in single data into a single unit , which is called object .
- Object function called member function, provides only way to access the data.
- You can't access data directly , data is hidden.
- Number of object communicate each other calling other's member function .



C++ Program

```
// C++ Program: Data and functions inside a class
#include <iostream>
using namespace std;

class BankAccount {
private:
   int balance;  // Data is protected

public:
   void deposit(int amount) {
      balance += amount;
   }

   void showBalance() {
      cout << "Balance: " << balance << endl;
   }
};
```

```
int main() {
   BankAccount acc;    // Object created
   acc.deposit(1000);  // Accessing function
   acc.showBalance();  // Output: Balance: 1000
   return 0;
}
```

- `balance` is a **private data member**.
- Only `deposit()` and `showBalance()` can access it.
- Data and behavior are **encapsulated** inside `BankAccount`.
- Improves **security** and **reusability**.

Feature	Structured Programming (C)	Object-Oriented Programming (C++)
Focus	Functions	Objects and Classes
Data and Function	Separate	Combined in one unit
Data Hiding	Not possible	Possible using access specifiers
Code Reusability	Low	High (using inheritance)
Real-world modeling	Difficult	Easy (object = real-world entity)
Example	int balance; void deposit();	class BankAccount { int balance; };

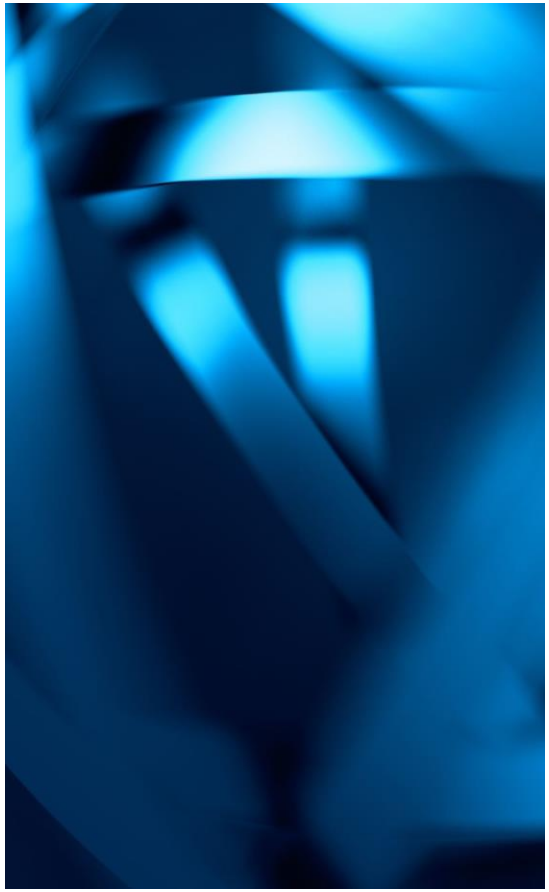
Feature	How it helps in large projects
Encapsulation	Protects internal logic and data
Modularity	Divide system into manageable parts
Reusability	Save time by reusing existing classes
Scalability	Add features without breaking other parts
Maintainability	Easier to fix and update
Real-world Mapping	Easier to understand and design systems

Encapsulation

OOP binds **data and functions** together inside classes.

Data is hidden (private/protected) and accessed only through member functions.

Prevents accidental data modification and improves security.



Modularity

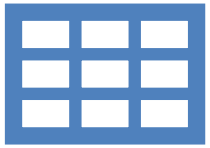
Large applications are divided into **independent classes or modules**.

Each class handles a specific part of the system.

Easier to develop, test, and debug each part separately.

Reusability

- Once a class is written, it can be **reused** in other projects.
- Inheritance allows one class to use properties and functions of another.
- Saves time and effort in writing duplicate code.



Scalability

- Easy to add new features or update existing ones.
- You can introduce new classes without modifying existing code.
- Ideal for **growing projects**.

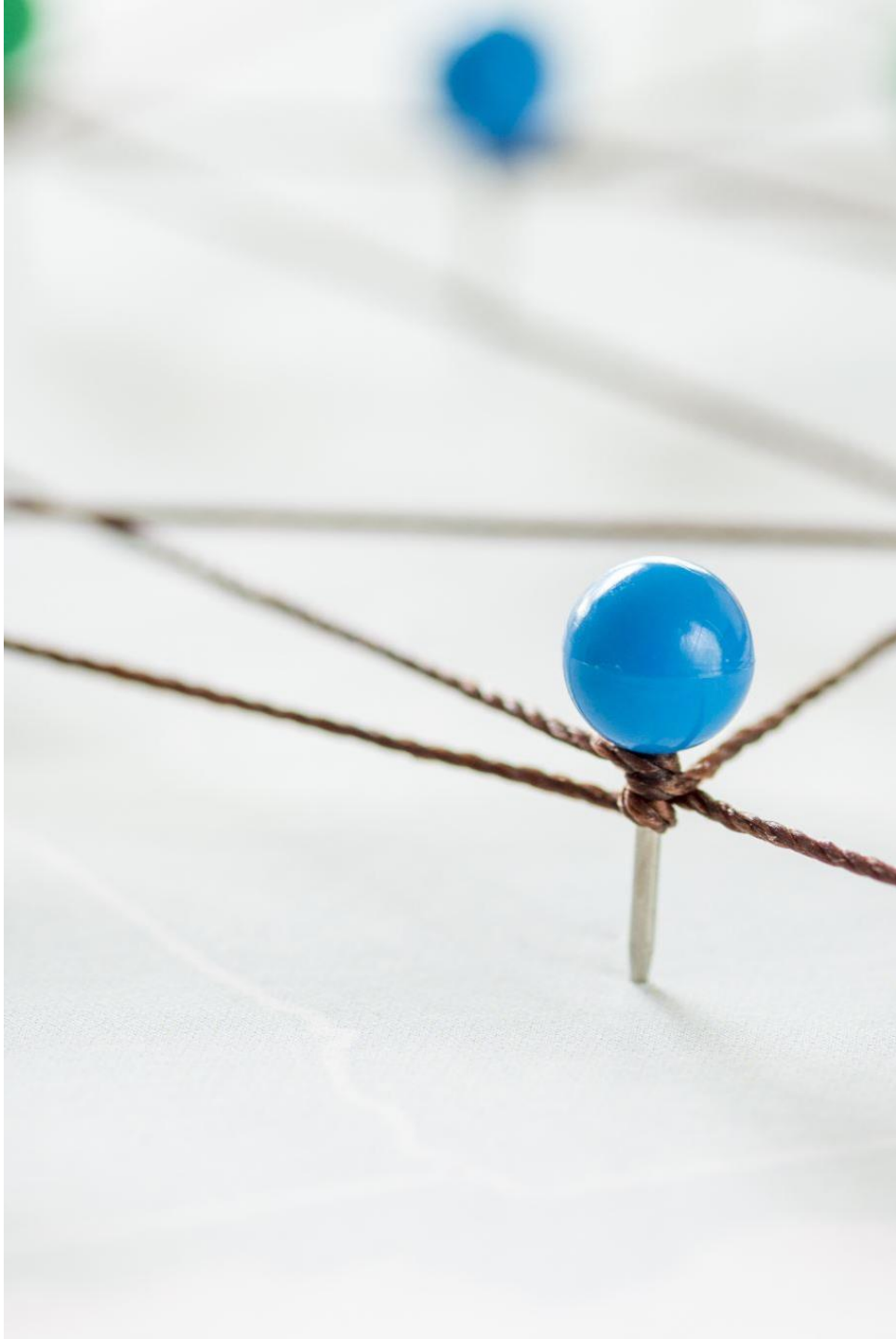
Maintainability

Changes or bug fixes can be made in isolated parts of the program.

Since classes are independent, updating one doesn't break others.

Real World Representation

- OOP allows direct modeling of **real-world entities** as objects.
- Makes program design more **intuitive and relatable**.



C vs C++

Basis of Comparison	C (Structured Programming)	C++ (Object-Oriented Programming)
Programming Paradigm	Procedural / Structured Programming	Object-Oriented Programming (OOP)
Approach	Top-down	Bottom-up
Data & Functions	Separate	Combined inside classes (Encapsulation)
Security	Less secure (global variables)	More secure (data hiding using access specifiers)
Code Reusability	Low	High (Inheritance and Classes)
Function Overloading	Not supported	Supported
Operator Overloading	Not supported	Supported
Inheritance	Not possible	Supported
Polymorphism	Not supported	Supported (Compile-time and Run-time)
Namespace Feature	Not available	Available to avoid name conflicts
Standard I/O	Uses printf() and scanf()	Uses cin and cout (also supports C I/O)
File Extension	.c	.cpp
Use Case	Small applications	Large applications and system-level programming

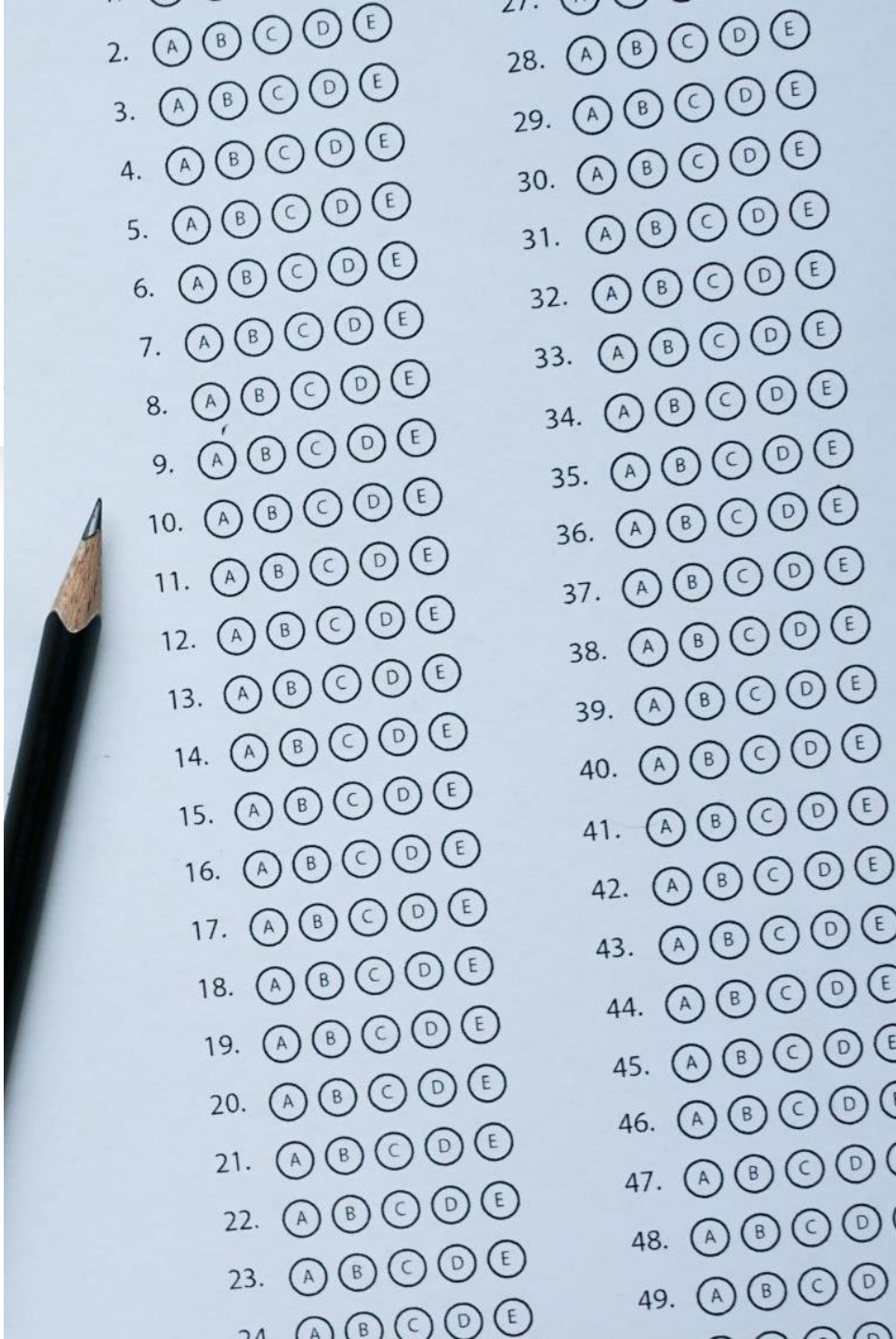
Summary

Object-Oriented Programming is better for large projects because it:

- Keeps code **organized and secure**
- Promotes **code reuse**
- Supports **easy debugging, testing, and scaling**
- Makes complex systems easier to understand and maintain

Homework

- Read Chapter one of OOP in C++ Pearson book .
- Complete page 25,26,27.



Overview of Topics

1.4 C++ Program Structure

1.5 Data Types, Variables and Constants

1.6 Insertion and Extraction Operators

1.7 Type Conversion

1.8 Structure in C++

1.9 Dynamic memory allocation : new and delete operator

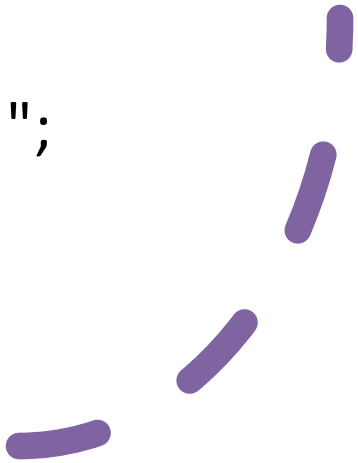
Basic C++ Program Structure

A typical C++ program includes:

- Preprocessor directives
- Main function
- Statements and expressions inside curly braces

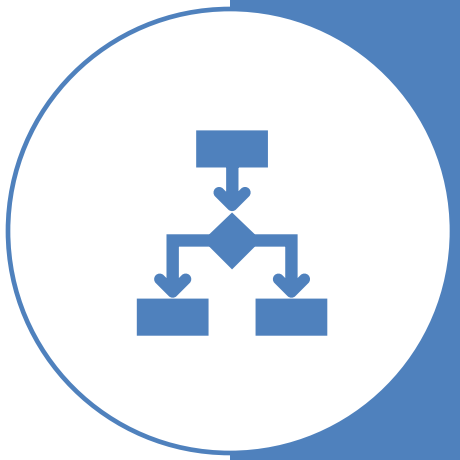
Example:

```
#include <iostream>
int main() {
   std::cout << "Hello, World!";
   return 0;
}
```



Preprocessor Directive

- A **preprocessor directive** in C++ is a command that starts with a # symbol and is processed **before the actual compilation** of the code begins.
- It tells the **C++ preprocessor** to perform specific actions like including files, defining constants, or conditionally compiling code.



Preprocessor Directives

- Begin with '#'
- Commonly used: #include, #define
- #include <iostream> includes standard input/output stream library
- #define is used for macros , compile time constant (#define PI 3.14159)

Common Preprocessor Directives

Directive	Purpose
#include	Includes the content of another file (usually header files).
#define	Defines macros or symbolic constants.
#undef	Undefines a macro defined with #define.
#ifdef / #ifndef	Checks if a macro is defined or not.
#if, #else, #elif, #endif	Conditional compilation.

Main Function and Execution Flow

- Entry point of every C++ program: int main()
- Code within main() executes sequentially
- Return value 0 signifies successful execution

Namespaces in C++

- A **namespace** is a way to group **related variables, functions, classes, etc.** under a single name to avoid **name conflicts** in large projects.
- Suppose two libraries have a function called `print()`. Without namespaces, the compiler won't know which one you mean. Namespaces **help avoid conflicts**.

Namespace

Defining

```
namespace MyNamespace {
    int value = 100;

    void show() {
        std::cout << "Value: " << value <<
std::endl;
    }
}
```

Using

1. Using Scope Resolution Operator : :

```
MyNamespace::show();
std::cout << MyNamespace::value;
```

2. Using Directive (`using namespace`)

```
using namespace MyNamespace;

show();        // No need to prefix with namespace
std::cout << value;
```

NameSpace (Function example)

```
#include <iostream>

namespace English {
    void greet() {
        std::cout << "Hello!" << std::endl;
    }
}

namespace Spanish {
    void greet() {
        std::cout << "¡Hola!" << std::endl;
    }
}

int main() {
    English::greet();  // Outputs: Hello!
    Spanish::greet();  // Outputs: ¡Hola!
    return 0;
}
```

Sample Program

```
#include <iostream>
using namespace std;

int main() {
    cout << "Welcome to C++";
    return 0;
}
```

- Include necessary library
- Uses namespace to avoid std::
- Outputs message using cout

Comments

Comments are used to explain code.

They do not affect the execution of the program.

Help improve code readability.

Single-line Comment:

Syntax: `// comment`

`Example: // This is a comment`

Multi-line Comment:

Syntax:

/* This is multi line comment */

Comments Best Practice

1

Use comments to explain *why* the code exists.

2

Avoid obvious comments.

3

Keep comments updated with code changes.

Data Types

A **data type** in programming defines **what kind of data** a variable can hold and **how much memory** it occupies.

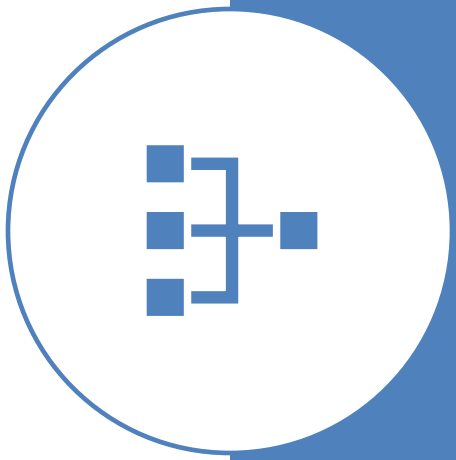
Why are Data Types Important?

- They tell the computer **how to interpret the bits** stored in memory.
- They help the compiler **check for errors** and **optimize memory use**.
- Different data types support **different operations** (e.g., math on numbers, or characters in text).

Built-in Data Types

C++ provides the following fundamental data types:

- int: Integer values (e.g., 10)
- float: Floating point numbers (e.g., 3.14)
- double: Double precision floating point
- char: Single character (e.g., 'A')
- bool: Boolean values (true or false)



Integer Data Types

Type	Size (Bytes)	Range
short	2	-32,768 to 32,767
unsigned short	2	0 to 65,535
int	4	-2,147,483,648 to 2,147,483,647
unsigned int	4	0 to 4,294,967,295
long	4 or 8	Platform-dependent
unsigned long	4 or 8	Platform-dependent
long long	8	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
unsigned long long	8	0 to 18,446,744,073,709,551,615

Floating Point Dat Types

Type	Size (Bytes)	Range (Approx.)
float	4	$\pm 1.2 \times 10^{-38}$ to $\pm 3.4 \times 10^{38}$ (6 digits precision)
double	8	$\pm 2.3 \times 10^{-308}$ to $\pm 1.7 \times 10^{308}$ (15 digits)
long double	8 or 16	Platform-dependent (higher precision)

Character and Boolean Types

Type	Size (Bytes)	Range
char	1	-128 to 127 (signed)
unsigned char	1	0 to 255
wchar_t	2 or 4	Wide character (Unicode)
char16_t	2	UTF-16 character
char32_t	4	UTF-32 character
bool	1	true (1) or false (0)

Variable

A **variable** is a **named storage** that holds data which can **change** during program execution.

You must **declare** the variable's **data type** before using it.

Variable names must follow identifier rules (e.g., no spaces, no starting with numbers)

Syntax

dataType variableName = value;

int age = 25;

float temperature = 36.6;

char grade = 'A';

You can **change** the value of a variable later:

age = 30; // age now holds 30

Constant

A **constant** is like a variable, but its value **cannot change** after initialization.

Declared using the `const` keyword.

Helps prevent accidental changes to important values.

Syntax

const dataType constantName = value;

const double PI = 3.14159;

const int DAYS_IN_WEEK = 7;

Trying to change a constant will cause a **compile-time error**:

PI = 3.14; // Error! Cannot assign to a constant

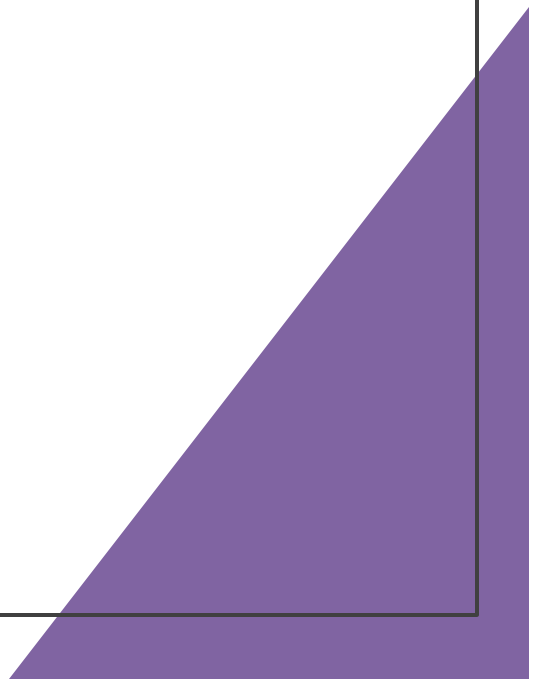
Constants in C++

- const keyword: const int size = 10;
- #define macro: #define SIZE 10
- Constants cannot be changed during execution
- Use constants to improve code readability and maintenance

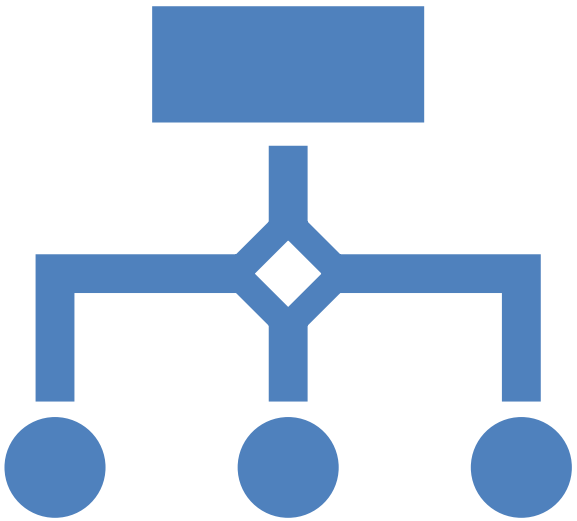
Example: Variables and Constants

```
const float PI = 3.14;
int radius = 5;
float area = PI * radius * radius;
cout << "Area: " << area;
```

- PI is a constant
- Variables used to compute area



Operators in C++



Operators are symbols that tell the compiler to perform specific mathematical, logical, or relational operations.

Types of Operators

- Arithmetic
- Relational
- Logical
- Assignment
- Incremental

Arithmetic Operator

Used to perform basic math calculations.

Operator	Meaning	Example	Result
+	Addition	5 + 3	8
-	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division	6 / 3	2
%	Modulus (remainder)	5 % 3	2

Relational Operator

- Used to compare two values, results in `true` or `false`.

OPERATOR	MEANING	EXAMPLE	RESULT
==	Equal to	5 == 3	false
!=	Not equal to	5 != 3	true
>	Greater than	5 > 3	true
<	Less than	5 < 3	false
>=	Greater or equal	5 >= 5	true
<=	Less or equal	3 <= 5	true

Logical Operators

- Used to combine multiple conditions.

Operator	Meaning	Example	Result
&&	Logical AND	(5 > 3) && (2 < 4)	true
||	Logical OR	(5>3) || (2>4)	true
!	Logical NOT	!(5 == 3)	true

Assignment Operators

Used to assign values to variables.

Operator	Meaning	Example	Result
=	Assign	x = 5	x = 5
+=	Add and assign	x += 3	x = x + 3
-=	Subtract and assign	x -= 2	x = x - 2
*=	Multiply and assign	x *= 4	x = x * 4
/=	Divide and assign	x /= 2	x = x / 2
%=	Modulus and assign	x %= 3	x = x % 3

Increment and Decrement Operators

Operator	Meaning	Example	Result
++	Increment by 1	x++ or ++x	x = x + 1
--	Decrement by 1	x-- or --x	x = x - 1

Escape Sequence

- Sometimes, you want your program to display special characters or format the text output in a certain way. Since you can't type these special commands directly in strings, C++ uses **escape sequences** to handle this.

- They are **special codes** inside strings that start with a backslash (\).
- They tell the program to perform an action or insert a special character.
- For example, moving to a new line, adding tabs, or printing quotes.

Common Escape Sequence

ESCAPE CODE	DESCRIPTION	EXAMPLE	OUTPUT
\n	Inserts a **new line**	cout << "Hi\nThere";	Hi There
\t	Inserts a **tab space**	cout << "A\tB";	A B
\\	Prints a **backslash**	cout << "\\";	\
\"	Prints a **double quote**	cout << "\"Hello\"";	"Hello"
\'	Prints a **single quote**	cout << "It\'s OK";	It's OK
\a	Produces a **beep sound**	cout << "\a";	(Beep sound if supported)
\b	Deletes the **previous character**	cout << "ABC\bD";	ABD
\r	Returns cursor to **start of line**	cout << "12345\rABC";	ABC45 (ABC overwrites start)

Why to use Escape Sequence

TO **FORMAT YOUR OUTPUT** CLEARLY (NEW LINES, TABS).

TO **INCLUDE SPECIAL CHARACTERS** IN YOUR TEXT.

TO MAKE YOUR PROGRAM'S OUTPUT **EASIER TO READ AND UNDERSTAND**.

Input/Output in C++

- cin: Standard input (keyboard)
- cout: Standard output (screen)
- Requires iostream library
- Used with insertion (<<) and extraction (>>) operators

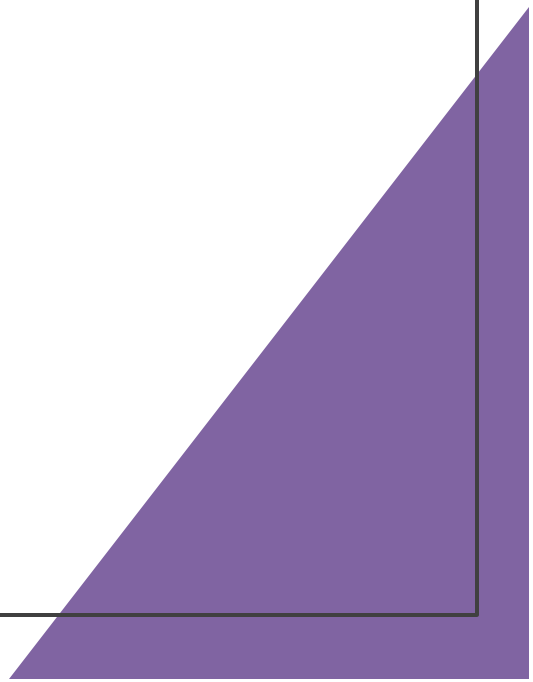
Insertion and Extraction Operators

<< : Outputs data to console (cout)

>> : Accepts input from user (cin)

Example:

```
int age;
cin >> age;
cout << "Age is: " << age;
```



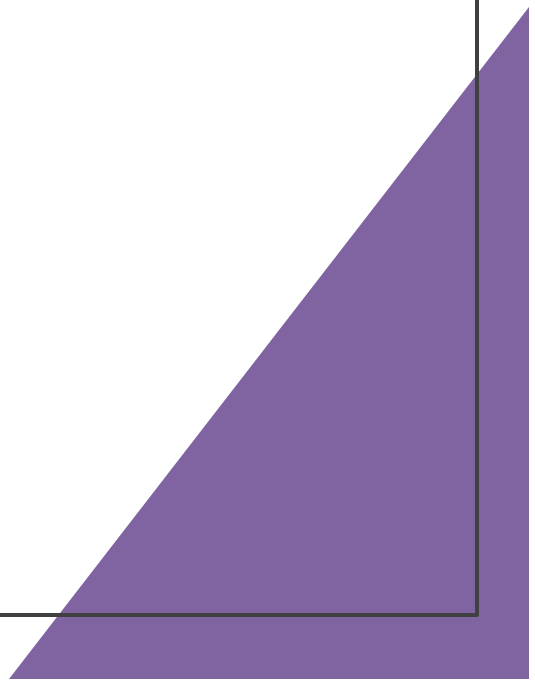
I/O Example Program

```
#include <iostream>
using namespace std;

int main() {
   int num;
   cout << "Enter a number: ";
   cin >> num;
   cout << "You entered: " << num;
   return 0;
}
```

What is Type Conversion?

- Converting data from one type to another
- Implicit: Done automatically by compiler
- Explicit: Manual casting by programmer



Implicit Type Conversion

- Occurs automatically during expressions
- Lower to higher type: int to float
- Example: int x = 10; float y = x; // x is promoted

Explicit Type Conversion

Also called type casting

Syntax: (data_type)variable;

Example: float a = 10.5; int b = (int)a; // b = 10

Type Conversion Example

```
int a = 5, b = 2;
float result = (float)a / b;
cout << "Result: " << result;
```

Casts a to float to get decimal result

endl

- `endl` stands for **end line**.
- It is used with **`cout`** to **insert a newline** character.
- It also **flushes the output buffer**, meaning it forces the output to be printed immediately.

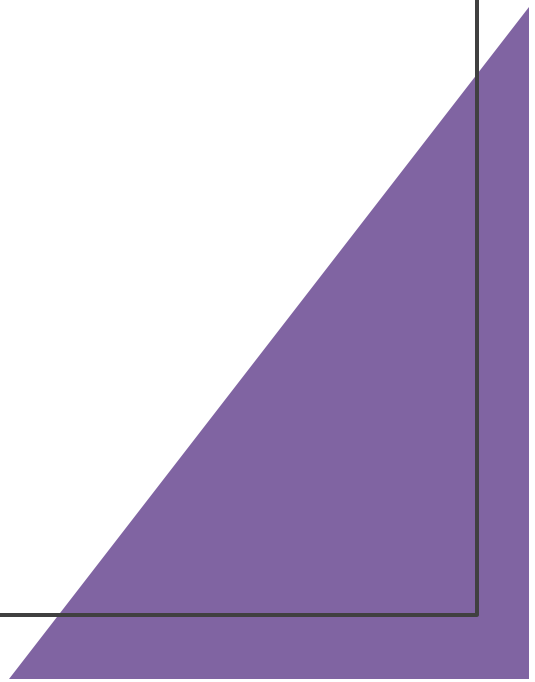
User-defined and Derived Types

User-defined types:

- struct: Combines variables under one type
- enum: Enumerated constants

Derived types:

- Arrays, pointers, references



Example

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello World" << endl;  // Moves to next line after printing
    cout << "Welcome to C++" << endl;
    return 0;
}
```

Structure

Structure is collection of simple and relatable variables, variables in structure can be of diff types , some can be int , some can be float and so on .

It helps represent real-world entities like a Student, Book,Employee, etc., where each item has multiple properties of different types.

Why to use it

- To group related data types (like `int`, `float`, `char[]`) together
- To keep code organized and readable
- To model complex data logically

Syntax

```
struct StructureName {
    dataType member1;
    dataType member2;
    ...
};
```

Structure

```
#include <iostream>
using namespace std;

struct Student {
    int id;
    float marks;
};

int main() {
    Student s;
    s.id = 1; //assigning values
    s.marks = 85.5; //assigning values

    cout << "Student ID: " << s.id << endl; //accessing
    cout << "Marks: " << s.marks << endl;

    return 0;
}
```

What is Dynamic Memory Allocation?

Dynamic memory allocation is the process of **allocating memory at runtime** (while the program is running), rather than at compile time.

This allows programs to request memory as needed, making efficient use of memory especially when the amount of data is not known beforehand.

Why use Dynamic Memory Application

- When you don’t know the exact size of data beforehand.
- To allocate memory **flexibly and efficiently**.
- To handle data that changes size during program execution.

Use operators `new` and `delete` to allocate and free memory dynamically.

Enter elements store in array and display

```
#include <iostream>
using namespace std;

int main() {
   int n;
   cout << "Enter number of elements: ";
   cin >> n;

   // Dynamically allocate array of size n
   int* arr = new int[n];

   cout << "Enter " << n << " elements:" << endl;
   for (int i = 0; i < n; i++) {
      cin >> arr[i];
   }

   cout << "You entered: ";
   for (int i = 0; i < n; i++) {
      cout << arr[i] << " ";
   }

   // Free the allocated memory
   delete[] arr;

   return 0;
}
```

Building a C++ program to record marks, and the number of students isn’t fixed.

```
#include <iostream>
using namespace std;

int main() {
   int n;
cout << "Enter number of students: ";
cin >> n;

int* marks = new int[n]; // dynamic array based on user input

for (int i = 0; i < n; ++i) {
   cout << "Enter marks for student " << i+1 << ": ";
   cin >> marks[i];
}

delete[] marks;  // Don't forget to free memory!


   return 0;
}
```

Summary of Topics

- C++ program starts with main()
- Variables and constants store data
- cin/cout used for I/O
- Type conversion changes data type
- Structures group related data
- Dynamic memory helps manage heap memory

Homework

- Read Chapter 2, 3 , 4 and 5 of Pearson book.
- Do question 1-25 of page 69,70 and 71.
- Do question no 7 of page 72.
- Do question 1-258 of page 125-126
- Write code for ques 1 and 2 , page 126.
- Do question 1-30 of chapter 5 page 210-212.